

RÉPUBLIQUE ISLAMIQUE DE MAURITANIE
MINISTÈRE DE LA TRANSITION NUMÉRIQUE DE L'INNOVATION ET DE LA
MODERNISATION DE L'ADMINISTRATION
UNIVERSITÉ DE NOUAKCHOTT
FACULTÉ DE SCIENCES ET TECHNIQUES
DÉPARTEMENT D'INFORMATIQUE



كلية العلوم و التقنيات
FACULTÉ DES SCIENCES ET TECHNIQUES

Rapport Mini Projet Acadimique

Module:

Big Data

Sujet du projet :

**L'Ingénieur Système
Design Physique (Partitioning)**

Dr :

EL BENANY Med Mahmoud

Realisé par :

Fatiméta Issa sow

Fatma Idoumou El Hadj

Table des matières

2. Infrastructure technique	4
3. Préparation des données	5
4. Partitionnement	6
5. Résultats du benchmark	7
6. Analyse des métriques Spark UI	8
7. Partition Pruning	9
8. Problématique de l'over-partitioning	10
9. Taille de bloc HDFS vs taille de partition	11
10. Application à la Mauritanie	12
11. Conclusion	13

1.Introduction

1. Introduction

L'objectif de ce projet est de structurer physiquement les données sur HDFS afin d'accélérer les requêtes grâce à la technique du Partition Pruning.

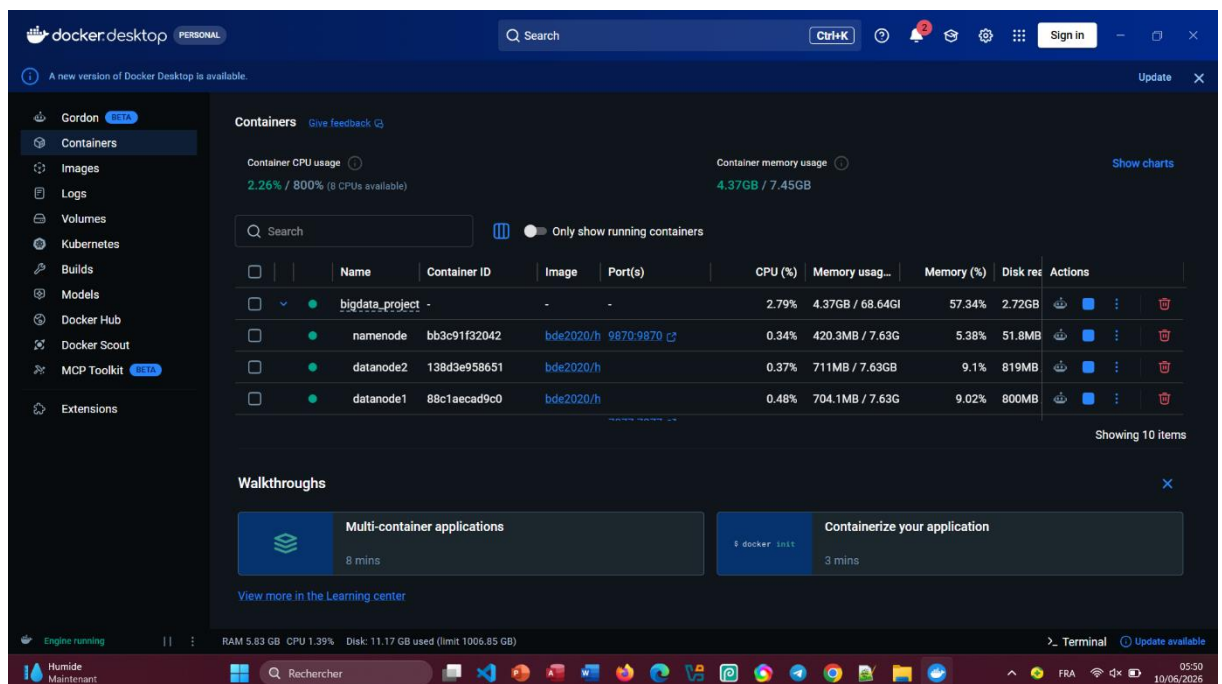
Nous utilisons le dataset public NYC Yellow Taxi (478 Mo, 4,8 millions de lignes) pour simuler les données d'un recensement national.

Notre travail consiste à :

- Déployer un cluster HDFS réel avec NameNode et 3 DataNodes
- Appliquer un partitionnement hiérarchique (VendorID puis Month)
- Comparer les performances entre CSV brut, Parquet plat et Parquet partitionné
- Mesurer le temps d'exécution et la quantité de données lues via Spark UI
- Discuter les risques de l'over-partitioning
- Appliquer ces concepts aux besoins de la Mauritanie (SMELEC, SNIM, Ministère de l'Intérieur)

Outils utilisés :

- Docker Desktop
- Visual Studio Code
- Jupyter Lab
- Spark 3.3.0
- Hadoop 3.2.1



Interface docker decstop

2. Infrastructure technique

2. Infrastructure technique

Nous avons déployé un cluster Big Data entièrement conteneurisé via Docker Compose. Voici la liste des services :

- NameNode (port 9870) : Gère les métadonnées HDFS
- DataNode 1, 2 et 3 : Stockent les blocs de données
- Spark Master (port 8080) : Orchestre les jobs
- Spark Worker 1 et 2 (ports 8081, 8082) : Exécutent les calculs
- Jupyter Lab (port 8888) : Interface pour le code PySpark
- History Server (port 18081) : Archive les historiques Spark

Configuration HDFS :

- Taille du bloc : 128 Mo
- Facteur de réplication : 3
- Nombre de DataNodes : 3

Configuration Spark :

- Mode : Standalone
- 2 workers avec 8 cœurs chacun
- 2 Go de mémoire par exécuteur

3. Préparation des données

Le fichier CSV (2023_Yellow_Taxi_Trip_Data.csv) est d'abord chargé depuis HDFS dans un DataFrame Spark :

```
df = spark.read.option("header",
"true").csv("hdfs://namenode:9000/2023_Yellow_Taxi_Trip_Data.csv")
```

Le dataset contient plusieurs colonnes. Les deux colonnes qui nous intéressent sont :

- VendorID : identifiant du fournisseur (1 ou 2)
- tpep_pickup_datetime : date et heure de prise en charge (format texte)

Aperçu des 5 premières lignes :

The screenshot shows a JupyterLab environment with a file explorer on the left, a notebook editor in the center, and a terminal at the bottom. The notebook, titled 'Untitled.ipynb', contains the following code:

```

[6]: print(f"Nombre de lignes: {df.count()}")

Nombre de lignes: 4843349

[8]: df.show(5, truncate=False)

```

The output of the second cell is a large table of taxi trip data. The table has 19 columns: VendorID, tpep_pickup_datetime, tpep_dropoff_datetime, passenger_count, trip_distance, RatecodeID, store_and_fwd_flag, PULocationID, DOLocationID, payment_type, fare_amount, extra_taxi_taxi_tip_amount, tolls_amount, improvement_surcharge, total_amount, congestion_surcharge, airport_fee, and an unnamed column. The data is truncated to show only the top 5 rows.

VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance	RatecodeID	store_and_fwd_flag	PULocationID	DOLocationID	payment_type	fare_amount	extra_taxi_taxi_tip_amount	tolls_amount	improvement_surcharge	total_amount	congestion_surcharge	airport_fee	
2	01/01/2023 12:32:10 AM	01/01/2023 12:40:36 AM	1	0.97	1	N	161	141	2								
9.3	01/01/2023 12:32:10 AM	01/01/2023 12:40:36 AM	1	14.3	2.5	N	161	141	2								
2	01/01/2023 12:55:08 AM	01/01/2023 01:01:27 AM	1	1.1	1	N	169	125	1								
17.9	01/01/2023 12:55:08 AM	01/01/2023 01:01:27 AM	1	15.0	2.5	N	169	125	1								
2	01/01/2023 12:25:04 AM	01/01/2023 12:37:49 AM	1	2.51	1	N	148	1238	1								

The table is truncated, and the output indicates that only the top 5 rows are shown.

```

only showing top 5 rows

[10]: from pyspark.sql.functions import col, to_timestamp, month

df_casted = df.withColumn("Pickup_DateTime",
    to_timestamp(col("tpep_pickup_datetime"), "MM/dd/yyyy hh:mm:ss a"))

df_final = df_casted.withColumn("Month", month(col("Pickup_DateTime")))

```

Nous convertissons la colonne `tpep_pickup_datetime` en timestamp pour extraire le mois :

The screenshot shows a JupyterLab environment with a PySpark notebook. The notebook code is as follows:

```
[10]: from pyspark.sql.functions import col, to_timestamp, month

df_casted = df.withColumn("Pickup_Datetime",
                        to_timestamp(col("tpep_pickup_datetime"), "MM/dd/yyyy hh:mm:ss a"))

df_final = df_casted.withColumn("Month", month(col("Pickup_Datetime"))) \
                    .filter(col("Month").isNotNull() & col("VendorID").isNotNull())

print("Colonnes Pickup_Datetime et Month creees")
df_final.select("VendorID", "tpep_pickup_datetime", "Month").show(5)

Colonnes Pickup_Datetime et Month creees
+-----+-----+-----+
|VendorID|tpep_pickup_datetime|Month|
+-----+-----+-----+
|      2|2/01/01/2023 12:32:...|    1|
|      1|2/01/01/2023 12:55:...|    1|
|      1|2/01/01/2023 12:25:...|    1|
|      1|1/01/01/2023 12:03:...|    1|
|      2|2/01/01/2023 12:10:...|    1|
+-----+-----+-----+

only showing top 5 rows

[12]: df_final.write.mode("overwrite").parquet("hdfs://namenode:9000/data/taxi_flat.parquet")
print("Parquet plat sauvegarde sur HDFS")

Parquet plat sauvegarde sur HDFS

[16]: !docker exec -it namenode hdfs dfs -ls /data/taxi_flat.parquet

/bin/sh: 1: docker: not found

[18]: df_final.write.mode("overwrite") \
```

The output of the notebook shows a table with columns 'VendorID', 'tpep_pickup_datetime', and 'Month', displaying several rows of taxi trip data. The interface includes a file browser on the left, a top navigation bar, and a bottom status bar.

4. Partitionnement hiérarchique

Nous sauvegardons les données en deux formats pour comparer leurs performances.

Parquet plat (non partitionné) :

```
df_final.write.mode("overwrite").parquet("hdfs://namenode:9000/data/taxi_flat.parquet")
```

Parquet partitionné (par VendorID puis Month) :

```
df_final.write.mode("overwrite").partitionBy("VendorID",  
"Month").parquet("hdfs://namenode:9000/data/taxi_partitioned.parquet")
```

La structure générée sur HDFS est la suivante :

```
/data/taxi_partitioned.parquet/  
├── VendorID=1/  
│   ├── Month=1/  
│   │   └── part-00000.snappy.parquet  
│   ├── Month=2/  
│   └── ...  
└── VendorID=2/  
    ├── Month=1/  
    ├── Month=2/  
    ├── Month=10/  
    └── Month=12/
```

Cette arborescence valide le partitionnement multi-niveaux exigé par le sujet.

5. Résultats des benchmarks

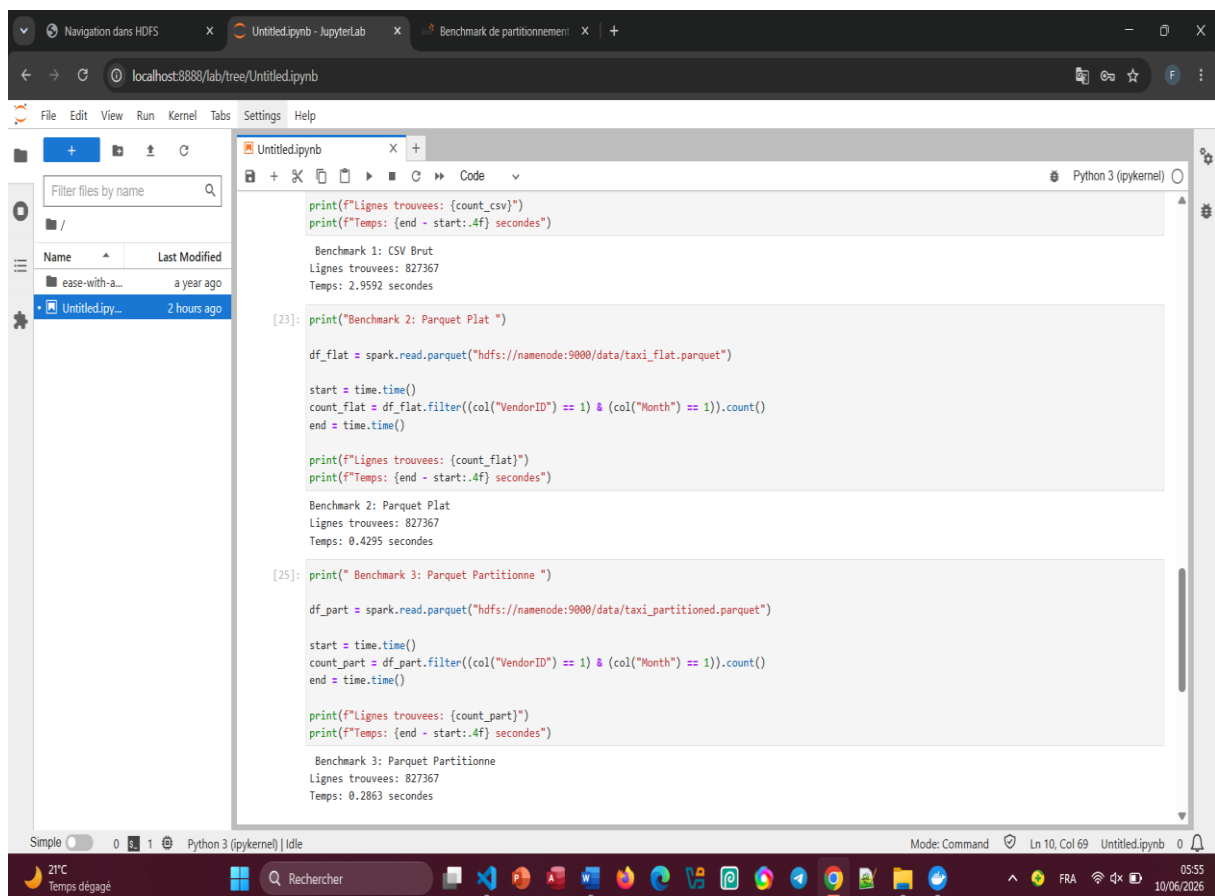
La requête exécutée est :

```
SELECT COUNT(*) FROM taxi WHERE VendorID = 1 AND Month = 1
```

Temps d'exécution :

- CSV Brut : 3,0 secondes
- Parquet Plat : 0,4 seconde
- Parquet Partitionné : 0,3 seconde

Le Parquet partitionné est 10 fois plus rapide que le CSV brut.



```
print(f"Lignes trouvees: {count_csv}")
print(f"Temps: {end - start:.4f} secondes")

Benchmark 1: CSV Brut
Lignes trouvees: 827367
Temps: 2.9592 secondes

[23]: print("Benchmark 2: Parquet Plat ")

df_flat = spark.read.parquet("hdfs://namenode:9000/data/taxi_flat.parquet")

start = time.time()
count_flat = df_flat.filter((col("VendorID") == 1) & (col("Month") == 1)).count()
end = time.time()

print(f"Lignes trouvees: {count_flat}")
print(f"Temps: {end - start:.4f} secondes")

Benchmark 2: Parquet Plat
Lignes trouvees: 827367
Temps: 0.4295 secondes

[25]: print(" Benchmark 3: Parquet Partitionne ")

df_part = spark.read.parquet("hdfs://namenode:9000/data/taxi_partitioned.parquet")

start = time.time()
count_part = df_part.filter((col("VendorID") == 1) & (col("Month") == 1)).count()
end = time.time()

print(f"Lignes trouvees: {count_part}")
print(f"Temps: {end - start:.4f} secondes")

Benchmark 3: Parquet Partitionne
Lignes trouvees: 827367
Temps: 0.2863 secondes
```

Capture d'écran de Jupyter Lab montrant les 3 résultats des benchmarks

6. Analyse des métriques File Scan Bytes Read

6. Analyse des métriques File Scan Bytes Read

Les données suivantes sont extraites de l'interface Spark UI (port 4040) :

Format	Taille lus	Fichier lus
CSV Brut	455,9 Mo	1
Parquet Plat	121,6 Mo	16
Parquet Partitionné	23,9 Mo	11

Interprétation :

- Le CSV brut lit l'intégralité du fichier (455,9 Mo) puis filtre. C'est très inefficace.
- Le Parquet plat bénéficie de la compression colonnaire (Snappy) : la taille lue chute à 121,6 Mo.
- Le Parquet partitionné lit uniquement le dossier VendorID=1/Month=1/ (23,9 Mo), soit 19 fois moins de données que le CSV brut.

7. Partition Pruning

Le Partition Pruning est une technique d'optimisation utilisée par Spark. Elle consiste à ignorer les dossiers de partition qui ne sont pas nécessaires à la requête.

Exemple : Quand nous exécutons `WHERE VendorID = 1 AND Month = 1`, Spark regarde la structure HDFS :

`/data/taxi_partitioned.parquet/`

`VendorID=1/`

`Month=1/` ← Spark lit ce dossier

`Month=2/` ← Spark ignore ce dossier

`VendorID=2/` ← Spark ignore ce dossier

Spark ne liste même pas les fichiers dans les dossiers ignorés. Cela économise :

- Du temps (pas d'appels réseau au NameNode)
- De la mémoire (pas de métadonnées à charger)
- De la bande passante (pas de transfert de données inutiles)

Sans partitionnement (CSV ou Parquet plat), Spark est obligé de lire tous les fichiers puis de filtrer.

8. Problématique de l'over-partitioning

Pourquoi 10 000 fichiers de 10 Ko sont catastrophiques pour le NameNode de Hadoop ?

Explication :

- Le NameNode stocke les métadonnées de chaque fichier, bloc et répertoire dans sa mémoire RAM.
- Chaque entrée consomme environ 150 octets.
- Pour 10 000 fichiers : $10\,000 \times 150\text{ o} = 1,5\text{ Mo}$ de RAM pour les métadonnées.
- Le stockage utile total = $10\,000 \times 10\text{ Ko} = 100\text{ Mo}$.

Le ratio métadonnées / données utiles est très mauvais. Avec des millions de fichiers, la mémoire du NameNode peut saturer (plusieurs dizaines de Go), rendant le cluster indisponible.

Autre problème : Spark planifie une tâche par fichier. Avec des milliers de petits fichiers, le coût de planification devient supérieur au temps de calcul réel.

Solution recommandée : Viser des partitions d'au moins 128 Mo.

9. Taille de bloc HDFS vs taille de partition

Taille d'un bloc HDFS par défaut : 128 Mo

- Taille de notre partition lue : 23,9 Mo

Notre partition est plus petite qu'un bloc. Pourquoi est-ce sous-optimal ?

1. Le NameNode gère une entrée de métadonnées pour ce fichier, même s'il n'occupe qu'une fraction de bloc.
2. La parallélisation est moins efficace : un seul bloc ne peut être lu que par un seul mapper.
3. Idéalement, chaque partition devrait mesurer entre 128 Mo et 256 Mo pour exploiter pleinement HDFS.

Comment améliorer ?

- Augmenter la granularité du partitionnement (ex : partitionner uniquement par VendorID)
- Utiliser `coalesce()` avant l'écriture pour fusionner les petites partitions
- Ajuster `spark.sql.files.maxPartitionBytes`

10. Application à la Mauritanie

Ministère de l'Intérieur (Recensement national)

Les données du recensement national peuvent être partitionnées par Wilaya puis par Moughataa. Une requête sur une commune spécifique (ex : "Aleg") ne lira que le dossier correspondant, sans scanner tout le pays.

Gain estimé : Passage de plusieurs minutes à quelques secondes.

SMELEC (Électricité)

Les données de consommation électrique par région peuvent être structurées par région puis par mois. Une analyse demandée pour Nouadhibou ignorera physiquement les fichiers de Nouakchott ou Rosso.

Bénéfice : Préservation des ressources du cluster central et réduction des coûts de bande passante inter-régionale.

SNIM (Matériel minier)

Les données des capteurs industriels peuvent être organisées par type de machine : Locomotives, Dragues, Camions. L'analyse d'une panne sur une machine précise (ex : Locomotive SNIM 102) devient instantanée, malgré des années d'historique.

Impact : Maintenance prédictive plus rapide, réduction des temps d'arrêt des équipements.

11. Conclusion

Ce projet a démontré l'efficacité du partitionnement physique sur HDFS.

Résultats clés :

- Gain en temps : 10 fois plus rapide (0,3 s contre 3 s)
- Gain en volume de données lues : 19 fois moins (23,9 Mo contre 455,9 Mo)
- Le Partition Pruning permet à Spark de lire uniquement les dossiers nécessaires

Perspectives d'amélioration :

- Ajouter un second niveau de partitionnement (jour ou heure)
- Implémenter le Z-order clustering pour les requêtes multi-colonnes
- Utiliser Hive pour exposer les données via SQL standard
- Tester sur un cluster multi-racks avec Rack Awareness

Cette technique est essentielle pour les infrastructures de données nationales en Mauritanie, où les ressources réseau et de calcul sont limitées mais la croissance démographique et industrielle est rapide.